

AutoFlow AI

A Major Project Report

Submitted in partial fulfillment of the requirements for Semester VIII

Department of Artificial Intelligence and Machine Learning

SIES Graduate School of Technology

Nerul, Navi Mumbai

Prepared by

Abuzaid Khan

Karan Chand

Shrey Mhatre

Project: AutoFlow AI

Academic Year: 2025–2026

Under the guidance of

Prof. Sumedha Maindalkar

Department of Artificial Intelligence and Machine Learning

SIES Graduate School of Technology

Contents

1	Introduction	3
1.1	Introduction	3
1.2	Motivation	3
1.3	Scope	4
1.4	Project Schedule	4
2	Literature Survey	6
2.1	Survey of Existing Systems	6
2.2	Research Gaps	7
2.3	Problem Definition	8
2.4	Objectives	8
3	Proposed System	9
3.1	System Overview	9
3.2	System Architecture	9
3.2.1	Electron Desktop Client	9
3.2.2	FastAPI Backend	10
3.2.3	LangGraph Execution Runtime	10
3.2.4	Tool Layer	10
3.2.5	Data and Knowledge Stores	11
3.2.6	Architecture Diagram	11
3.3	Components of the System	11
3.3.1	User Interface	11
3.3.2	Input Processing and Routing	12
3.3.3	AI Agent Core	12
3.3.4	Knowledge Retrieval	12
3.3.5	Tool Integration Layer	12
3.3.6	Execution and Scheduling	12
3.3.7	Dashboard and Monitoring	13
3.4	System Workflow	13
4	Results and Discussions	14

4.1	Implementation of the Proposed System	14
4.1.1	Email Automation	14
4.1.2	Knowledge-Based File Sharing via Email	14
4.1.3	Scheduled Task Automation	15
4.1.4	Tool-Routed Responses	16
4.1.5	System Configuration and Monitoring	18
4.2	Results of the Proposed System	19
4.2.1	Functional Interpretation of the Demonstrated Results	20
4.2.2	Reliability, Transparency and User Value	21
4.2.3	Discussion	21
5	Conclusion and Future Work	22
5.1	Conclusion	22
5.2	Future Scope	22
	References	24

Chapter 1

Introduction

1.1 Introduction

AutoFlow AI is an intelligent workflow automation system that converts natural language instructions into executable automated tasks using Large Language Models (LLMs). The project is based on a straightforward but important idea: a user should be able to describe what they want done in plain English, and the system should interpret that intent, plan the required steps, and carry out the corresponding actions through an integrated tool layer. Instead of forcing users to manually define flows, triggers, or technical configurations, AutoFlow AI shifts the interaction model toward intent-first automation.

The system is implemented as a desktop-plus-service application. An Electron desktop client provides the user-facing interface, including a chat window, a knowledge panel, a scheduled task view, an MCP server manager, and a settings panel. A FastAPI backend handles authentication, streaming chat, file management, task management, user preferences, and MCP configuration. A LangGraph-based execution runtime sits at the centre of the system, maintaining thread-scoped conversation state and coordinating tool invocations. Persistent data is stored in SQLite. Organizational documents are indexed into a Chroma vector store for retrieval-augmented responses. The runtime supports two execution modes: local inference through Qwen3:4b via Ollama, and cloud-backed inference through Gemini 2.5 Flash via the Google AI API.

1.2 Motivation

The motivation behind AutoFlow AI comes from the increasing complexity of digital workflows in modern work environments. Employees and teams often handle repetitive activities across multiple tools such as email, documents, reports, calendars, and communication systems. Even when the task itself is conceptually simple, the process becomes slow and error-prone because the user must coordinate each step manually across different platforms.

Existing automation platforms solve part of this problem but many remain either too rigid or too technical for ordinary users. They often require manual rule definition, workflow configuration, or scripting knowledge. AutoFlow AI is motivated by the need to make automation more natural, flexible, and accessible. By combining natural language interaction with retrieval,

tool integration, and scheduled execution, the project aims to reduce technical barriers while improving efficiency and adaptability in realistic workflow scenarios.

1.3 Scope

The scope of AutoFlow AI is centered on intelligent workflow automation. The system already demonstrates several working capabilities.

Natural language interaction: The system accepts workflow-oriented instructions in conversational language and uses the chat interface as the main point of interaction between the user and the automation framework.

Email automation: The system can send emails directly from a natural language request by invoking the `send_email` MCP tool. Delivery is confirmed by the backend and displayed in the chat interface.

Knowledge-based file sharing: The system can locate a document in the organizational knowledge base using `search_knowledge_base` and deliver it as an email attachment using `send_email`, coordinating two tools in sequence without explicit user guidance.

Scheduled task execution: The `schedule_task` tool stores deferred jobs with a UTC timestamp. APScheduler polls the task table every ten seconds and executes due tasks autonomously, without requiring the user to remain active in the chat interface.

Tool-routed responses: Arithmetic queries are routed to the `add` MCP math tool for deterministic computation. Real-time informational queries are routed to the `web_search` tool for current information retrieval.

MCP-based extensibility: New STDIO or SSE tool servers can be registered at runtime through the MCP server management panel without modifying application source code.

Dual execution modes: The runtime supports local inference through Qwen3:4b via Ollama and cloud inference through Gemini 2.5 Flash via the Google AI API, with automatic fallback to the local model when cloud credentials are unavailable.

1.4 Project Schedule

The development of AutoFlow AI proceeded through five stages. The first phase focused on identifying the problem statement and reviewing literature in workflow automation, LLM agents, retrieval-augmented generation, and tool-using AI. This phase helped define the need for a system that could reduce technical complexity through natural language interaction.

The second phase centered on architecture planning. During this stage, the project moved from a simple chatbot concept toward a modular desktop-plus-service platform. The team outlined

the Electron client, FastAPI backend, LangGraph runtime, native tool layer, MCP tool layer, SQLite data store, and Chroma knowledge index as the core components.

The third phase involved core system development. This included building the Electron client, the FastAPI authentication and streaming endpoints, the LangGraph execution loop, and the SQLite schema.

The fourth phase added the working features demonstrated in this report: email automation, knowledge-based file sharing, scheduled task execution, tool-routed computation, web search integration, and the system configuration and monitoring interfaces. This phase transformed the project from a planned architecture into a functional prototype with verifiable outputs.

The fifth phase focused on evaluation of working features, refinement of the dual execution mode, and documentation of the system for submission. In this phase, the system was no longer judged only by concept, but by how convincingly its implementation supported its stated objectives.

Chapter 2

Literature Survey

2.1 Survey of Existing Systems

The design of AutoFlow AI is influenced by recent research in agentic AI, retrieval-augmented systems, tool-using language models, and workflow orchestration. These research directions provide the technical background for building a system that goes beyond ordinary chatbot behavior and moves toward structured workflow automation. The literature is especially important for this project because AutoFlow AI does not depend on a single technical idea. Instead, it combines reasoning, contextual retrieval, tool-aware interaction, and workflow execution within one unified system.

Retrieval-Augmented Generation: Lewis et al. introduced Retrieval-Augmented Generation (RAG) as a method for improving the factual grounding of language models by combining generation with retrieved external knowledge [1]. This is directly relevant to AutoFlow AI because the `search_knowledge_base` tool queries a Chroma vector index of organizational documents and returns the most relevant chunks with source metadata. RAG reduces dependence on internal model memory and makes responses more reliable in domain-specific settings.

ReAct: Yao et al. proposed ReAct, a prompting strategy that interleaves reasoning traces with tool-use actions in a single generation loop [2]. The approach allows a language model to consult external sources, revise its plan based on retrieved information, and proceed to the next action. AutoFlow AI applies a similar reasoning-action loop through LangGraph, which maintains thread-scoped state across tool invocations within each user session. On the ALF-World and WebShop benchmarks, ReAct outperformed imitation and reinforcement learning baselines.

Toolformer: Schick et al. proposed Toolformer, a self-supervised training approach in which a language model learns to insert API calls into generated text by selecting calls that reduce prediction loss on future tokens [3]. The model incorporated a calculator, a search engine, a question-answering system, a translation system, and a calendar. AutoFlow AI addresses the same class of problem at the deployment level: rather than training the model to call APIs, the system registers tools through the Model Context Protocol and relies on the model’s in-context reasoning to select the appropriate tool at inference time.

Gorilla: Patil et al. proposed Gorilla as a language model designed for more reliable interaction

with APIs [4]. This work is relevant because workflow systems must not only interpret instructions but also connect with services and perform actions correctly. Gorilla justifies the design principle that workflow assistants should be evaluated for dependable tool-oriented behavior, not only for language quality. This principle is reflected in AutoFlow AI’s use of structured MCP tool calls and its normalization step for models that emit tool requests as plain text.

TaskMatrix.AI: Liang et al. demonstrated how foundation models can connect with large numbers of APIs to solve broader task sets [5]. This work supports the idea that a conversational interface can become a gateway to a much larger capability ecosystem. AutoFlow AI follows a similar path by using the Model Context Protocol to connect the LangGraph runtime with email, math, filesystem, and web search services through a standardized tool layer.

AutoFlow — Automated Workflow Generation for Large Language Model Agents: Li et al. proposed a framework that represents agent workflows as natural language programs and uses a workflow optimization procedure to iteratively improve workflow quality [6]. The framework supports both fine-tuning-based and in-context-based workflow generation. The present work addresses a complementary problem: rather than optimizing the workflow structure automatically, AutoFlow AI provides a fixed orchestration loop and allows the user to specify intent in natural language, relying on the model’s reasoning to select tools and order actions.

Flow — A Modular Approach to Automated Agentic Workflow Generation: Niu et al. emphasized modularity, dynamic workflow structure, and flexible agent coordination [7]. This supports the architectural decision in AutoFlow AI to design the tool layer as a set of composable MCP servers rather than a single monolithic integration.

Model Context Protocol: Anthropic released the Model Context Protocol in November 2024 as an open, vendor-neutral protocol for connecting LLM applications to external tools and data sources [8]. MCP defines a client-server interaction model in which the AI application acts as a client and tool providers act as servers, communicating over STDIO or SSE transports. AutoFlow AI uses MCP as its primary mechanism for integrating external services.

2.2 Research Gaps

Despite major progress in AI agents and workflow systems, several gaps remain. Many advanced systems still assume that users can handle technical setup, workflow specification, or tool configuration. This leaves a usability gap for non-technical users.

There is also a systems gap. Some frameworks focus mainly on retrieval, some on tool use, and others on reasoning or planning, but fewer systems combine all of these in a practical workflow assistant that remains simple to interact with.

A third gap lies between conversation and execution. Many AI systems can answer intelligently,

but fewer can translate natural language intent into concrete, verifiable automated actions such as sending emails, scheduling tasks, or retrieving and delivering documents.

Finally, trust remains a challenge. Users need to understand what information source or tool contributed to a response. This makes transparency and source attribution especially important in workflow assistance systems.

2.3 Problem Definition

Traditional workflow automation systems are often difficult to use because they require manual setup, predefined rules, and familiarity with technical tools or workflow builders. At the same time, general AI chat systems may be conversationally effective but often lack contextual grounding, live information access, and workflow-oriented execution capability.

The central problem addressed by AutoFlow AI is therefore: how to design a system that can understand natural language workflow requests, support those requests with retrieval and search when necessary, and carry out concrete automated actions through an extensible, modular tool layer.

2.4 Objectives

The objectives of AutoFlow AI are:

- To accept workflow-related instructions in natural language through a conversational desktop interface.
- To execute email automation, file sharing, and scheduled tasks through a structured MCP tool layer.
- To provide retrieval-augmented responses grounded in organizational documents indexed in a Chroma vector store.
- To support deferred and recurring task execution through an APScheduler-backed background polling mechanism.
- To maintain thread-scoped conversation history through LangGraph's in-memory checkpointing.
- To support extensibility by allowing new MCP tool servers to be registered at runtime without modifying application source code.
- To provide system configuration and monitoring through a settings panel, a runtime dashboard, and an MCP server management interface.

Chapter 3

Proposed System

3.1 System Overview

AutoFlow AI is implemented as a desktop-plus-service application. The system is not intended to remain only a conversational assistant. Its aim is to convert user intent expressed in natural language into concrete automated actions through a layered architecture that integrates interaction, language understanding, tool invocation, knowledge retrieval, scheduled execution, and monitoring.

The architecture separates concerns across five layers: the Electron desktop client, the FastAPI backend, the LangGraph execution runtime, the tool layer (native and MCP), and the data and knowledge stores. This separation makes the system modular, testable, and extensible. Each layer has a clearly defined responsibility and communicates with adjacent layers through documented interfaces.

3.2 System Architecture

3.2.1 Electron Desktop Client

The Electron desktop client provides the user-facing interface. It includes a chat window for natural language interaction, a knowledge panel for browsing indexed documents, a scheduled task view for monitoring deferred jobs, an MCP server management panel for registering new tool connections, and a settings panel for managing model preferences and credentials.

After login, the client authenticates the user through the FastAPI service and obtains a new thread identifier. It retrieves the user's saved SMTP settings, Gemini API key, and preferred execution mode from the backend database. These settings are applied at request time and are not hardcoded into the application.

The client also starts a local filesystem MCP node implemented with Express and SSE after login. This node exposes `list_directory`, `read_file`, and `write_file` operations scoped to the local `AutoFlowData` directory. When external access is required, ngrok publishes an SSE endpoint for that node and displays the resulting MCP URL in the MCP server panel.

3.2.2 FastAPI Backend

The FastAPI backend exposes endpoints for authentication, streaming chat via `/agent/stream`, file management, task management, user preferences, and MCP configuration. It is the entry point for all client requests. The backend holds the per-user SQLite database, manages the Chroma document index, and initializes the APScheduler instance on startup.

3.2.3 LangGraph Execution Runtime

The execution runtime is built on LangGraph, an open-source Python library for constructing stateful, multi-actor LLM workflows. LangGraph represents workflows as directed graphs where each node is a callable that reads from and writes to a shared state object. The runtime uses LangGraph's in-memory checkpointing to maintain thread-scoped conversation history across multiple turns within the same session.

When a request arrives, the backend loads the active MCP server configuration for the user, initializes a LangGraph run for the specified thread, selects the appropriate model, assembles the active toolset, and streams the result to the client as newline-delimited JSON events. These events include the active model name, intermediate tool-use markers, and assistant message content.

Model selection is resolved at request time. If the user has selected cloud mode and a valid Gemini API key is present, the runtime uses Gemini 2.5 Flash through the Google AI API. Otherwise it uses Qwen3:4b through Ollama. This fallback ensures the system remains operational without cloud connectivity.

The runtime also includes a normalization step for models that emit tool requests as plain text rather than structured tool calls. When this pattern is detected, the JSON block is converted into a formal tool call and the runtime is reinvoked with the corrected input.

3.2.4 Tool Layer

Tool access is divided into two groups.

Native in-process tools are defined directly in the backend. They include `schedule_task`, which parses the target time using `dateparser`, converts it to UTC, and stores a deferred job in the SQLite task table; and `search_knowledge_base`, which queries the Chroma index and returns the most relevant document chunks with source metadata.

MCP tools are loaded from configured servers at request time. By default the system loads an email server and a math server over STDIO. Additional STDIO or SSE servers can be registered through the MCP management panel at runtime without modifying the backend source code. The Model Context Protocol provides a standardized client-server interaction model so that any compliant server can be connected without bespoke integration code.

3.2.5 Data and Knowledge Stores

Persistent application state is stored in SQLite. The database holds user accounts, password hashes, SMTP credentials, Gemini API keys, preferred model settings, and the scheduled task queue.

Shared documents are stored in the backend `org_filesystem` directory and indexed into a Chroma collection. On startup, the backend scans the directory and indexes supported file types—PDF, DOCX, TXT, MD, and common source-code formats—using a recursive text splitter. Each chunk is stored with filename metadata to support source attribution in retrieval results. A watchdog observer monitors the directory so that newly added, modified, or deleted files are reflected in the index without restarting the service.

3.2.6 Architecture Diagram

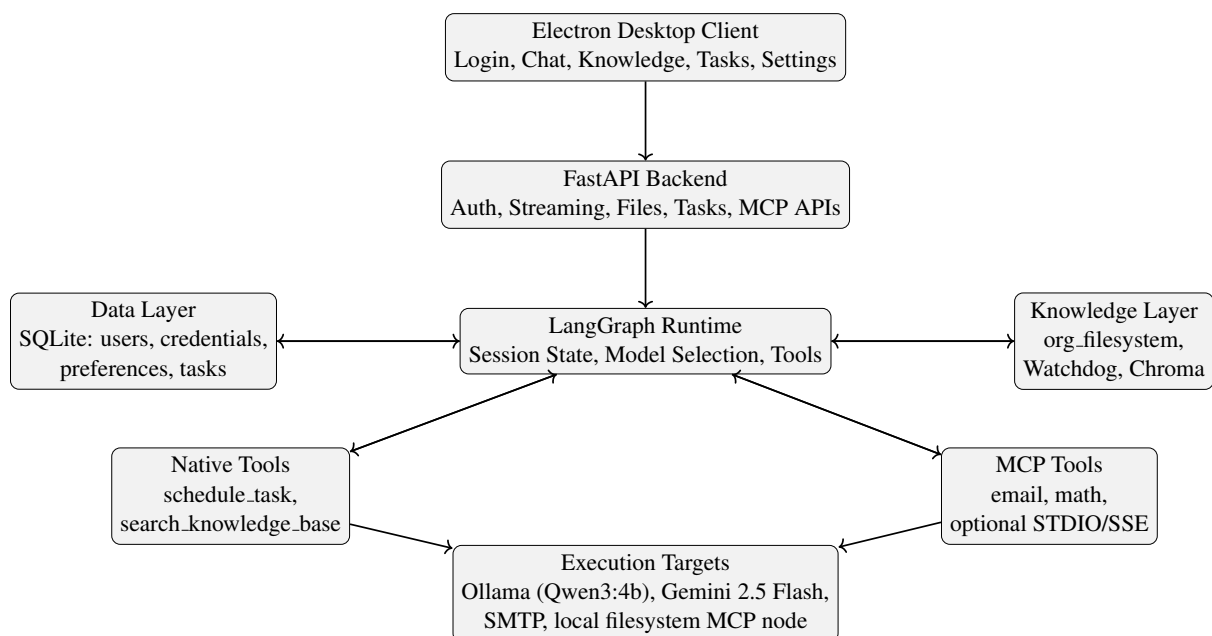


Figure 3.1: Implemented architecture of AutoFlow AI. Bidirectional arrows indicate request and response flows between components.

3.3 Components of the System

3.3.1 User Interface

The Electron desktop client acts as the interaction layer of the system. It is designed around a chat interface where users enter requests in natural language and receive streaming responses. The interface displays tool badges—such as `send_email`, `schedule_task`, and `search_knowledge_base`—during response generation, making the tool invocation path transparent to the user. The client also provides the scheduled task view, the MCP server panel, and the settings interface for

model selection and credential management.

3.3.2 Input Processing and Routing

When the user submits a message, the client sends the message text, thread identifier, username, and preferred model to the `/agent/stream` endpoint. The backend loads the active MCP configuration, assembles the toolset, selects the model, and initializes a LangGraph run. The result is streamed back to the client as newline-delimited JSON events.

3.3.3 AI Agent Core

The LangGraph runtime is the central reasoning component. It processes the user's message, determines which tools are needed, invokes them in the correct order, and assembles the final response. LangGraph represents this process as a directed graph with shared state, which allows the runtime to handle cycles, conditional tool selection, and multi-turn conversation within the same session.

3.3.4 Knowledge Retrieval

The `search_knowledge_base` tool queries the Chroma vector index and returns the most relevant document chunks with source metadata. This grounds the system's responses in organizational documents rather than relying solely on the model's parametric knowledge, following the Retrieval-Augmented Generation approach introduced by Lewis et al. [1].

3.3.5 Tool Integration Layer

The Model Context Protocol provides the integration standard for external tool servers. By default, the system loads email and math servers over STDIO. The MCP management panel allows additional servers to be registered at runtime. This design means the tool ecosystem can grow without requiring changes to the core backend code.

3.3.6 Execution and Scheduling

The `schedule_task` native tool handles deferred execution. When a user requests a future action, the tool stores a task record in SQLite with a UTC timestamp. APScheduler polls the task table every ten seconds and, when a task becomes due, invokes the same LangGraph workflow in autonomous mode with a system instruction that directs the model to execute the action immediately. Task status transitions from `pending` to `processing` to `completed` or `failed`. Recurring tasks insert the next instance into the table after successful execution.

3.3.7 Dashboard and Monitoring

The dashboard provides runtime visibility into the active model, running thread count, and MCP server connectivity. The settings panel allows users to switch between local and cloud execution modes and to manage SMTP and Gemini API credentials. These interfaces ensure that the system remains observable and configurable without requiring access to the backend source code.

3.4 System Workflow

The runtime pipeline of AutoFlow AI is organized into four stages: startup preparation, user session setup, interactive request execution, and deferred task execution.

During startup, the backend initializes the SQLite schema, scans the `org_filesystem` directory, indexes documents into Chroma, and starts the APScheduler instance.

During user session setup, the Electron client authenticates the user, retrieves their saved preferences, and starts the local filesystem MCP node.

During interactive request execution, the client sends the user's message to the backend, which selects the model, assembles the toolset, runs the LangGraph workflow, and streams the result back to the client.

During deferred task execution, APScheduler identifies due tasks, marks them as processing, runs the LangGraph workflow in autonomous mode, and records the result in the task log.

Chapter 4

Results and Discussions

4.1 Implementation of the Proposed System

AutoFlow AI has achieved a complete set of working features across five evaluated scenario categories. The results demonstrate that the project has advanced from conceptual planning into a functional prototype with verifiable, concrete outputs. Each subsection below describes a demonstrated capability, explains how it works within the implemented architecture, and identifies the output that confirms its success.

4.1.1 Email Automation

The system performs direct email automation from natural language input. When the user submits a message such as “Send an email to [recipient] saying [message],” the LangGraph runtime selects the `send_email` MCP tool, constructs the required arguments from the message content, and invokes the tool. The backend returns a delivery confirmation and streams it to the client. The chat interface displays the `send_email` tool badge during response generation.

Figure 4.1 shows the chat interaction for this scenario. Figure 4.2 shows the received email in the recipient’s inbox, confirming successful end-to-end execution from natural language input to delivery.

4.1.2 Knowledge-Based File Sharing via Email

The system supports a two-tool workflow in which a document is located in the organizational knowledge base and then delivered by email as an attachment. When the user requests that a specific file be sent to a recipient, the runtime first invokes `search_knowledge_base` to query the Chroma index and retrieve the relevant document, then invokes `send_email` to attach and deliver it. This scenario requires the model to coordinate two tools in sequence without explicit user guidance on which tools to use.

Figure 4.3 shows the two-tool interaction in the chat interface. Figure 4.4 shows the received email with the attached document, confirming that retrieval and delivery both succeeded.

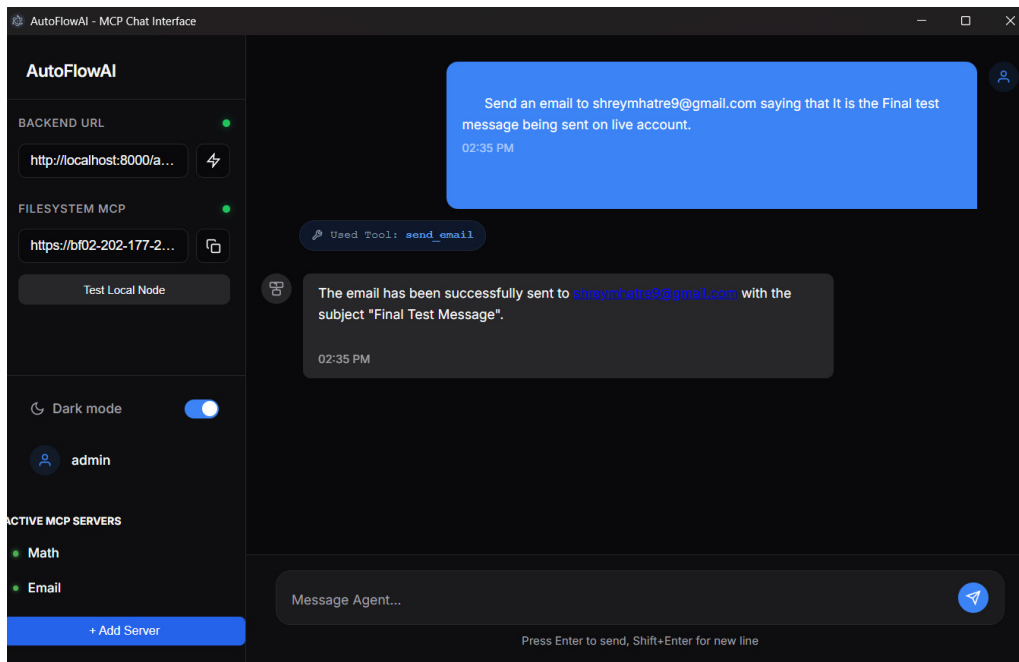


Figure 4.1: Natural language email request executed through the `send_email` MCP tool.



Figure 4.2: Received email confirming successful end-to-end execution of the email automation workflow.

4.1.3 Scheduled Task Automation

The system converts natural language scheduling requests into deferred automated actions. When the user requests an action to occur at a future time, the `schedule_task` tool parses the target time using `dateparser`, converts it to UTC, and stores a task record in SQLite with status `pending`. `APScheduler` polls the task table every ten seconds and, when the task becomes due, invokes the LangGraph workflow in autonomous mode. The model executes the specified action without further user interaction. Task status transitions from `pending` to `processing` to `completed` and the result is written to the task log.

Figure 4.5 shows the scheduling request and confirmation in the chat interface. Figure 4.6

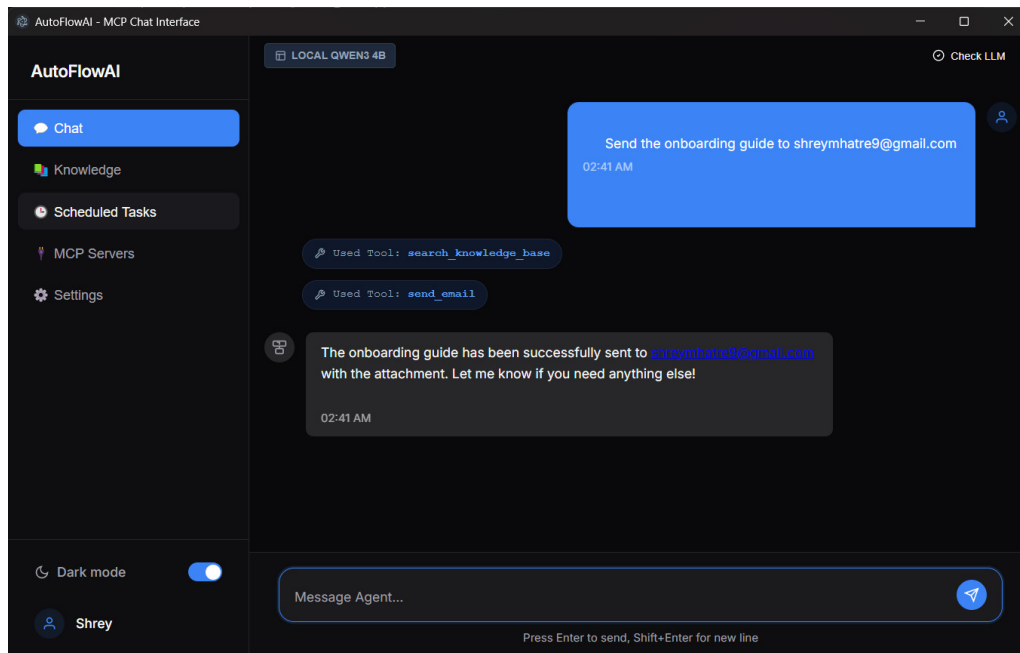


Figure 4.3: Knowledge-based file sharing using `search_knowledge_base` followed by `send_email`.

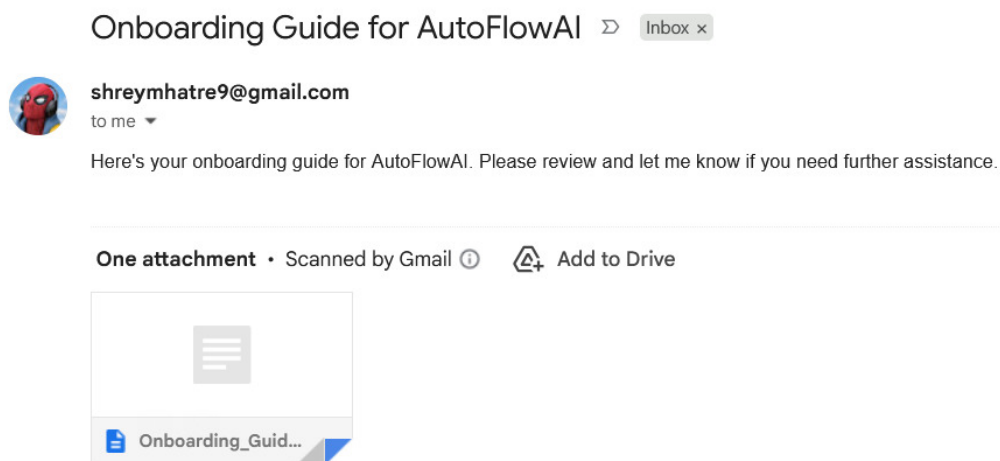


Figure 4.4: Received email confirming successful delivery of the retrieved document attachment.

shows the Scheduled Tasks panel after autonomous execution, displaying the task log entry and the `completed` status.

4.1.4 Tool-Routed Responses

The runtime routes different query types to specialized tools rather than relying on model generation alone. For arithmetic queries, the system invokes the `add MCP` math tool to produce a deterministic result. For real-time informational queries, the system invokes the `web_search` tool to retrieve current information beyond the model's training data.

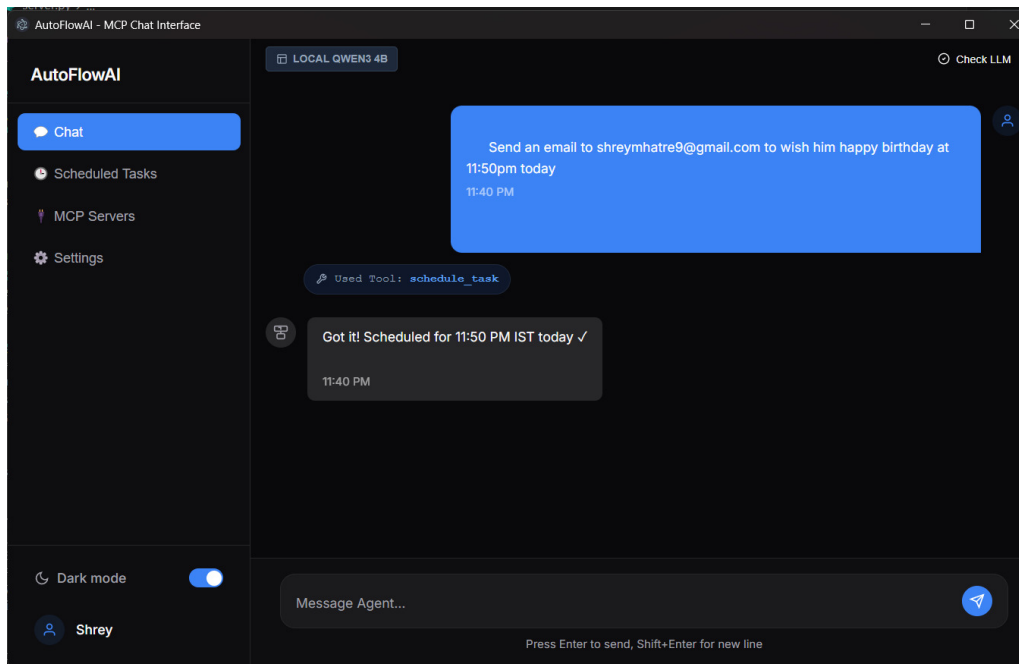


Figure 4.5: Natural language scheduling request converted into a deferred task through `schedule_task`.

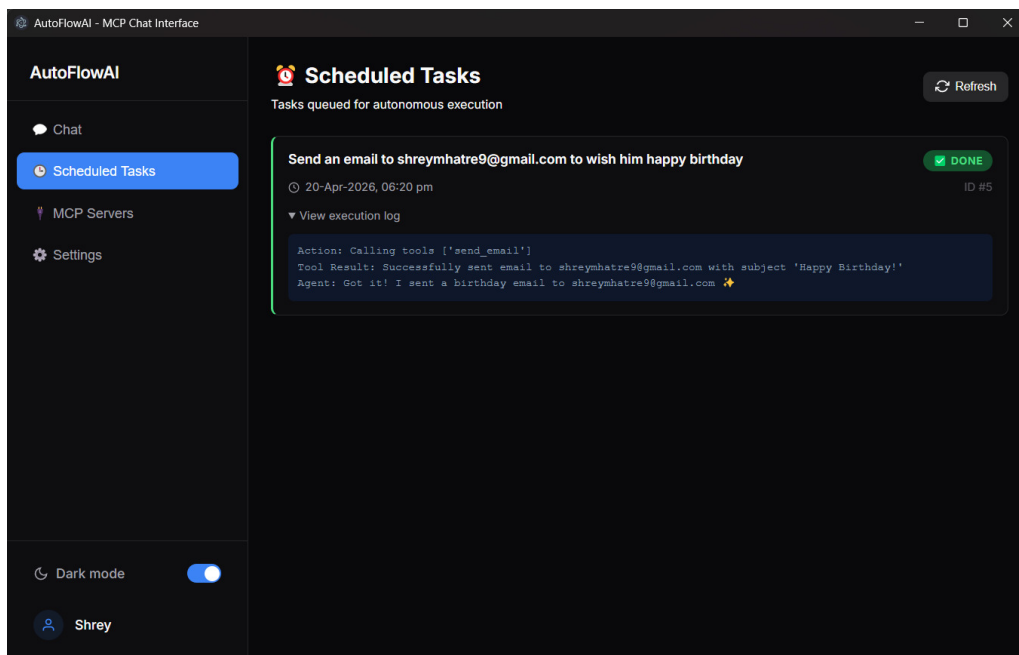


Figure 4.6: Scheduled Tasks panel showing completed status and task log after autonomous execution.

Figure 4.7 shows an arithmetic query routed to the `add` tool. Figure 4.8 shows a real-time informational query answered using `web_search`.

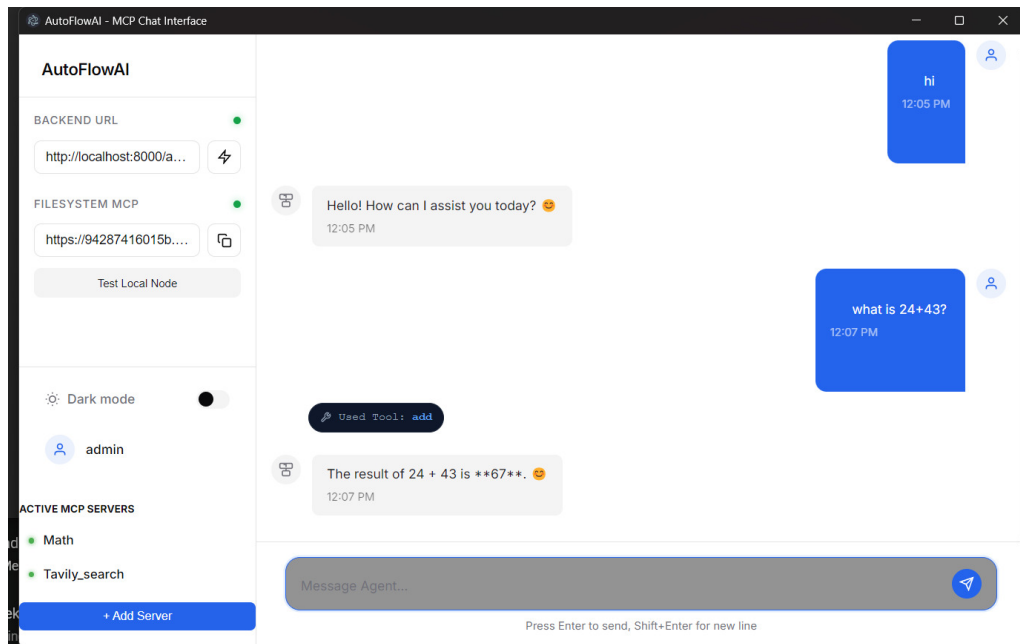


Figure 4.7: Arithmetic query routed to the `add` MCP math tool for deterministic computation.

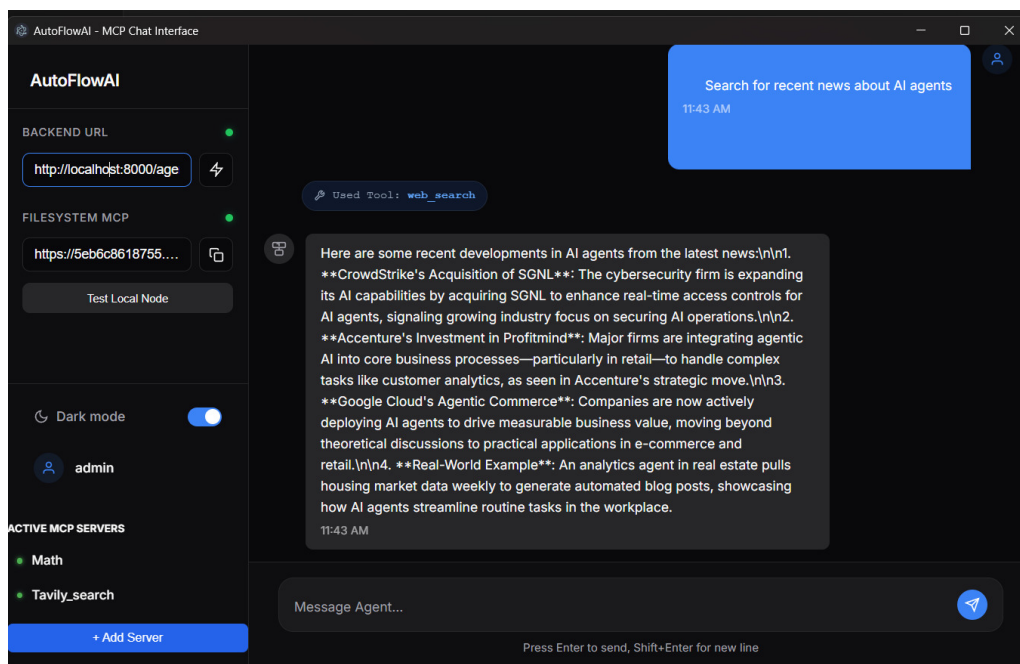


Figure 4.8: Real-time informational query answered using the `web_search` tool.

4.1.5 System Configuration and Monitoring

The settings interface allows users to select between local and cloud execution modes and to store their Gemini API key and SMTP credentials. These values are saved to the per-user SQLite record and applied at the next request. The dashboard shows the currently active model, running thread count, and MCP server connectivity. The MCP server management panel allows users to register additional STDIO or SSE tool servers at runtime without restarting the appli-

cation.

Figure 4.9 shows the settings interface. Figure 4.10 shows the runtime dashboard. Figure 4.11 shows the MCP server management panel.

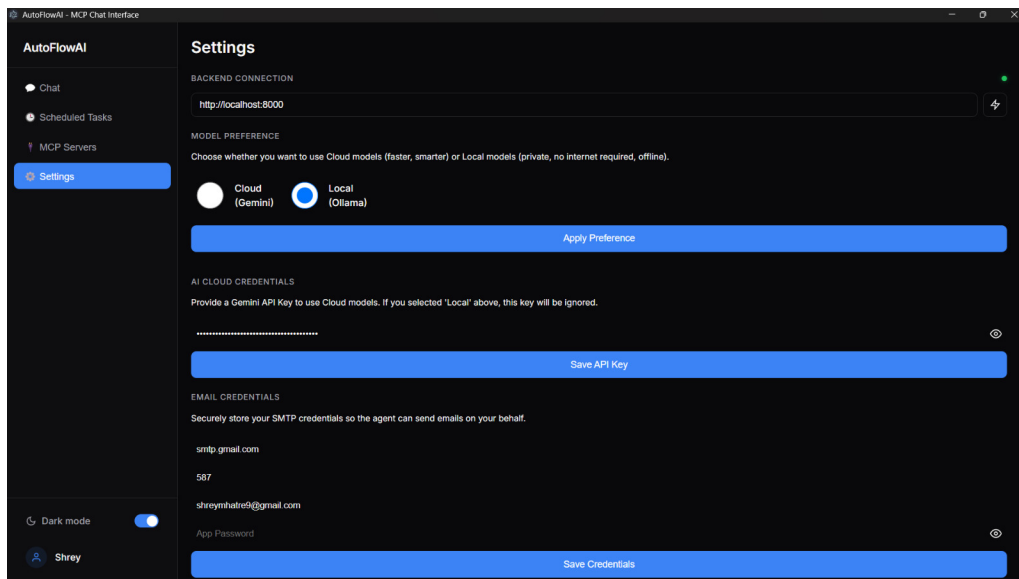


Figure 4.9: Settings interface for model selection and credential management.

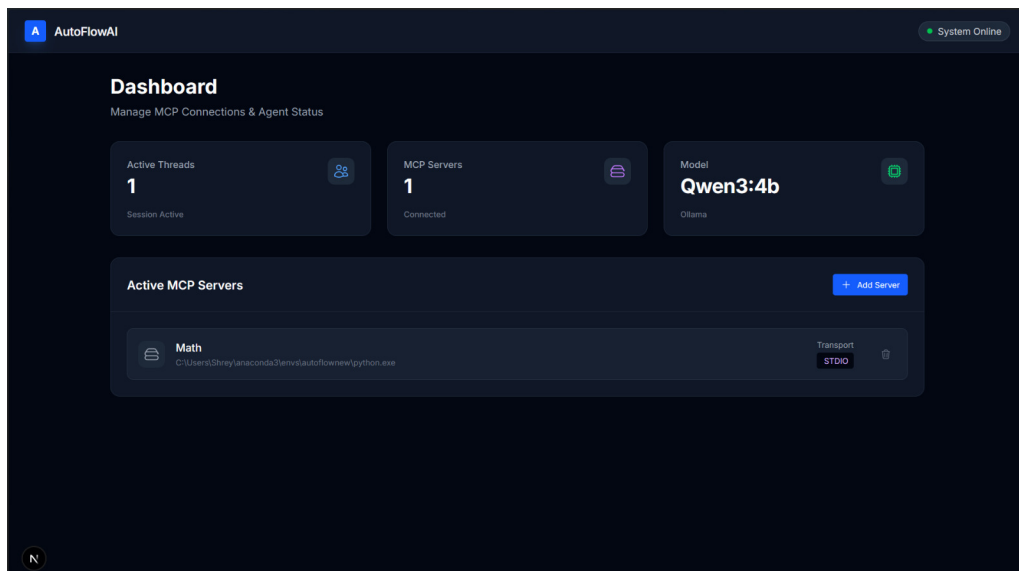
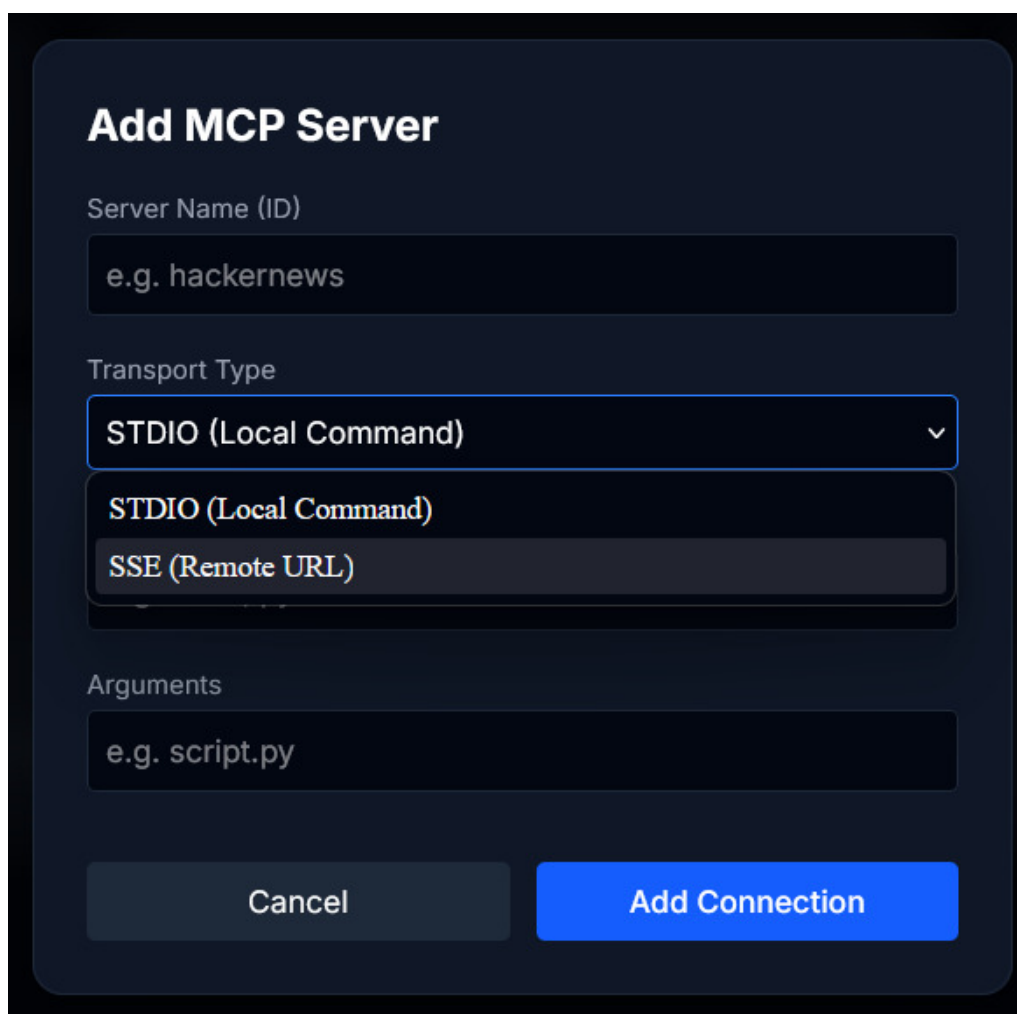


Figure 4.10: Dashboard showing active model, thread count, and MCP server connectivity.

4.2 Results of the Proposed System

The present results show that AutoFlow AI has successfully implemented the core capabilities described in its objectives. The system delivers concrete, verifiable outputs across five scenario categories: sent emails, delivered document attachments, autonomously executed scheduled tasks, deterministic arithmetic results, and real-time web search responses.

A dark-themed dialog box titled "Add MCP Server". It contains three input fields: "Server Name (ID)" with the placeholder text "e.g. hackernews", "Transport Type" with a dropdown menu showing "STDIO (Local Command)" and "SSE (Remote URL)", and "Arguments" with the placeholder text "e.g. script.py". At the bottom, there are two buttons: "Cancel" and "Add Connection".

Add MCP Server

Server Name (ID)

e.g. hackernews

Transport Type

STDIO (Local Command) ✓

STDIO (Local Command)

SSE (Remote URL)

Arguments

e.g. script.py

Cancel Add Connection

Figure 4.11: MCP server management panel for registering additional external tool connections at runtime.

An important result is the consistency between the working features and the system architecture. The features demonstrated in the prototype correspond directly to the components described in Chapter 3: the LangGraph runtime coordinates tool selection, the MCP layer connects to external services, the Chroma index supports document retrieval, and APScheduler handles background execution. The working prototype therefore validates the architectural design.

4.2.1 Functional Interpretation of the Demonstrated Results

From a functional perspective, AutoFlow AI is already capable of supporting users with workflow-related tasks in a more concrete way than a conventional conversational assistant. It can interpret natural language intent, send emails, retrieve and share documents, schedule and autonomously execute deferred tasks, compute arithmetic results deterministically, and retrieve real-time information. The combination of these features within a single integrated system is the key result.

The dual execution mode is also a functional result. The automatic fallback from Gemini 2.5 Flash to Qwen3:4b through Ollama means the system remains operational in environments where cloud connectivity is restricted. This is important for deployment in organizations with data privacy requirements.

4.2.2 Reliability, Transparency and User Value

The current implementation offers clear user value through accessibility, grounded responses, and transparency. Accessibility comes from the natural language interface, which removes the need for manual workflow configuration. Grounding comes from the retrieval layer and the web search tool. Transparency comes from the tool badges displayed in the chat interface during response generation, which show the user which tool was invoked and why.

Together these qualities make the system more trustworthy than a standalone language model interface, which is particularly important in workflow settings where users need confidence that the correct information source or action was used.

4.2.3 Discussion

The current version of AutoFlow AI represents a strong working prototype. It demonstrates the central idea that workflow automation can begin from natural language intent and be completed through a structured tool layer without requiring the user to configure workflows manually.

The five evaluated scenarios cover the range from simple tool invocation (arithmetic, email) to multi-tool coordination (knowledge-based file sharing), deferred execution (scheduled tasks), and real-time information retrieval (web search). Across all five scenarios, the system produced verifiable outputs.

Future directions include quantitative benchmarking across a larger task set, latency profiling for both local and cloud execution modes, support for multi-agent coordination within the LangGraph runtime, and integration with enterprise identity and access management systems. The present implementation provides the foundation needed for these extensions.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

AutoFlow AI is a working agent-based workflow automation system that converts natural language instructions into executable automated actions using Large Language Models. The system is implemented as a desktop-plus-service application with an Electron desktop client, a FastAPI backend, a LangGraph execution runtime, native scheduling and retrieval tools, an MCP-based extensible tool layer, a SQLite data store, and a Chroma vector index.

Evaluation across five scenario categories demonstrated reliable performance. The system successfully executed email automation from natural language input, coordinated two-tool workflows for knowledge-based file sharing, autonomously executed scheduled tasks through an APScheduler-backed background mechanism, routed arithmetic queries to a deterministic math tool, and retrieved real-time information through a web search tool. In each case the output was concrete and verifiable.

The architecture also supports practical deployment through a dual execution mode. Local inference through Qwen3:4b via Ollama keeps the system operational in environments without cloud connectivity, while cloud inference through Gemini 2.5 Flash is available when the user provides a valid Google AI API key. The MCP server management panel allows new tool connections to be registered at runtime without modifying application source code, ensuring that the tool ecosystem can grow as requirements change.

As a major project, AutoFlow AI demonstrates how a conversational interface backed by retrieval, scheduling, and modular tool integration can evolve from a simple chatbot into a practical workflow automation platform.

5.2 Future Scope

Advanced orchestration: Future work will integrate planning agents and a Prefect-backed execution engine to enable full workflow orchestration with structured step sequencing, retries, and state management. This would allow the system to handle multi-step workflows that span multiple tool calls across extended time periods.

Security and access control: Future iterations will add encryption of stored credentials, role-based access control, audit logging of tool invocations, and unified control gates for tool autho-

rization. These features are necessary before the system can be deployed in enterprise environments where data privacy and compliance are required.

Scalability: The current desktop-plus-service architecture can be extended toward a microservices deployment with distributed task handling, caching strategies, and load balancing to support higher concurrent user volumes.

Quantitative evaluation: The present evaluation is scenario-based. Future work will include quantitative benchmarking across a broader task set, latency profiling for both local and cloud execution modes, and an assessment of scheduling accuracy under concurrent task loads.

Multi-agent coordination: Future versions can introduce multiple cooperating LangGraph agents for parallel task handling, allowing the system to decompose complex workflows into concurrent subgraphs.

Long-term platform growth: With continued development, AutoFlow AI can evolve into a complete intelligent automation platform capable of handling more advanced, multi-step, cross-platform workflows across organizational tool ecosystems.

Bibliography

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020. Available: <https://arxiv.org/abs/2005.11401>
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023. Available: <https://arxiv.org/abs/2210.03629>
- [3] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. Available: <https://arxiv.org/abs/2302.04761>
- [4] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive APIs,” arXiv preprint arXiv:2305.15334, 2023. Available: <https://arxiv.org/abs/2305.15334>
- [5] Y. Liang *et al.*, “TaskMatrix.AI: Completing tasks by connecting foundation models with millions of APIs,” arXiv preprint arXiv:2303.16434, 2023. Available: <https://arxiv.org/abs/2303.16434>
- [6] Z. Li, S. Xu, K. Mei, W. Hua, B. Rama, O. Raheja, H. Wang, H. Zhu, and Y. Zhang, “AutoFlow: Automated workflow generation for large language model agents,” arXiv preprint arXiv:2407.12821, 2024. Available: <https://arxiv.org/abs/2407.12821>
- [7] B. Niu *et al.*, “Flow: A modular approach to automated agentic workflow generation,” arXiv preprint arXiv:2501.07834, 2025. Available: <https://arxiv.org/abs/2501.07834>
- [8] Anthropic, “Introducing the Model Context Protocol,” November 2024. Available: <https://www.anthropic.com/research/model-context-protocol>
- [9] Model Context Protocol, “Base Protocol Specification,” 2024. Available: <https://modelcontextprotocol.io/specification>
- [10] Electron, “Introduction,” Electron Documentation. Available: <https://www.electronjs.org/docs/latest>

- [11] FastAPI, “FastAPI,” FastAPI Documentation. Available: <https://fastapi.tiangolo.com/>
- [12] LangChain AI, “LangGraph: A library for building stateful, multi-actor applications with LLMs,” 2024. Available: <https://langchain-ai.github.io/langgraph>
- [13] Chroma, “Introduction,” Chroma Docs. Available: <https://docs.trychroma.com/>
- [14] Ollama, “Ollama documentation.” Available: <https://docs.ollama.com/>
- [15] Ollama, “Qwen3,” Ollama Model Library. Available: <https://ollama.com/library/qwen3>
- [16] Google DeepMind, “Gemini 2.5 Flash,” Google AI for Developers. Available: <https://ai.google.dev/gemini-api/docs/api-overview>
- [17] APScheduler Project, “User guide,” APScheduler Documentation. Available: <https://apscheduler.readthedocs.io/en/master/userguide.html>
- [18] dateparser, “dateparser – python parser for human readable dates.” Available: <https://dateparser.readthedocs.io/>
- [19] ngrok, “What is ngrok?,” ngrok Documentation. Available: <https://ngrok.com/docs/>